

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Мобильная разработка

Отчет
Курсовая работа

Выполнил:

Якунин Андрей

Группа

К3441

Проверил:

Шатинский Г.С.

Санкт-Петербург
2026 г.

Отчёт по курсовой работе

1. Тема

Разработка Android-приложения «Мессенджер» с использованием встроенной навигации и архитектуры MVVM

Стек: Kotlin, Android SDK, AndroidX, Navigation Component, ViewModel и LiveData, Room, Retrofit, Material Components for Android.

2. Задача 1 - навигация по вкладкам

ТЗ

В рамках работы требуется разработать простое Android-приложение «Мессенджер» с одной главной Activity и навигацией на основе Fragment. Навигация между экранами реализуется с помощью Bottom Navigation и компонента Navigation. Приложение должно содержать экраны новостей, профиля и настроек. Также необходимо реализовать логирование жизненного цикла Activity и Fragment.

Решение

MainActivity - главная Activity с Bottom Navigation

Navigation Graph - настроена навигация между 3 экранами

Три фрагмента:

FeedFragment (новостная лента - заглушка)

ProfileFragment (профиль пользователя с данными)

SettingsFragment (переключатель темной/светлой темы)

NavController - управление навигацией через Navigation Component

Логирование жизненного цикла - Log.d() в onCreate, onStart, onResume, onPause, onStop, onDestroy для Activity и всех Fragments

3. Задача 2 - MVVM и управление состоянием профиля и настроек

ТЗ

Необходимо реализовать архитектуру MVVM для управления состоянием приложения. Данные профиля пользователя и настройки темы должны храниться во ViewModel и сохраняться при повороте экрана. Экран профиля позволяет редактировать имя и статус, а экран настроек - переключать тему оформления. Обновление интерфейса выполняется реактивно через LiveData с логированием жизненного цикла ViewModel

Решение

1) Использование ViewModel для хранения состояния.

Для экранов профиля и настроек реализованы отдельные ViewModel, в которых хранятся данные пользователя и выбранные параметры оформления. Такой подход позволяет вынести логику работы с данными за пределы UI, оставив интерфейсу только отображение состояния и обработку пользовательских действий.

2) Экран профиля пользователя.

На экране профиля отображаются имя и статус пользователя, которые можно изменять с помощью полей ввода. Введённые значения сохраняются во ViewModel, а интерфейс автоматически обновляется при изменении данных благодаря использованию LiveData. Это обеспечивает корректную работу экрана при повороте устройства и пересоздании фрагмента.

3) Экран настроек и управление темой.

В разделе настроек реализован переключатель светлой и тёмной темы. Выбранное значение сохраняется во ViewModel и применяется ко всему приложению через AppCompatActivity, что демонстрирует работу MVVM для хранения и применения глобальных настроек.

4) Сохранение состояния при изменении конфигурации.

Так как все данные хранятся во ViewModel, при повороте экрана состояние приложения сохраняется. Пользовательские данные и выбранные настройки не теряются, что положительно влияет на удобство использования приложения.

5) Логирование жизненного цикла ViewModel.

В ViewModel добавлено логирование основных этапов жизненного цикла - создания и уничтожения объектов. Это позволяет контролировать корректность работы архитектуры MVVM и отслеживать момент освобождения ресурсов.

4. Задача 3 - Загрузка сообщений из API и Room для офлайн доступа

ТЗ

Необходимо реализовать загрузку списка сообщений из внешнего API с последующим сохранением данных в локальную базу Room для обеспечения офлайн-доступа. Сетевое взаимодействие должно выполняться с использованием Retrofit и корутин. Отображение сообщений требуется реализовать на экране «Лента» с помощью RecyclerView. Репозиторий должен выступать единой точкой доступа ViewModel к данным из сети и базы

Решение

1) Репозиторий сообщений.

В проекте реализован класс репозитория, отвечающий за получение и хранение сообщений. Он объединяет работу с удалённым API и локальной базой данных Room, предоставляя ViewModel единый механизм доступа к данным. Такой подход позволяет централизовать логику работы с источниками данных и реализовать принцип единого источника истины.

2) Сетевое взаимодействие через Retrofit и корутины.

Для обращения к внешнему API используется Retrofit, настроенный на работу с корутинами. Это обеспечивает асинхронную загрузку данных без блокировки главного потока и упрощает обработку сетевых ответов и возможных ошибок

3) Отображение ленты сообщений.

Список сообщений выводится на экране «Лента» с использованием RecyclerView. Каждый элемент списка содержит основную информацию о сообщении, что позволяет пользователю удобно просматривать загруженный контент.

4) Локальное хранение данных.

После успешного получения сообщений из сети данные сохраняются в локальную базу данных Room. Это позволяет повторно использовать уже загруженную информацию и снижает количество сетевых запросов.

5) Поддержка офлайн-режима.

При отсутствии подключения к интернету репозиторий автоматически загружает данные из локальной базы. Такой механизм обеспечивает стабильную работу приложения и доступность информации в офлайн-режиме

6) Ручное обновление данных.

На экране ленты реализована кнопка обновления, которая инициирует повторную загрузку данных через ViewModel. При этом данные обновляются как в интерфейсе, так и в локальной базе, предоставляя пользователю контроль над актуальностью информации

5. Задача 4 - Улучшение интерфейса и фоновой синхронизации

ТЗ

Требуется доработать пользовательский интерфейс приложения и расширить его функциональность за счёт внедрения элементов Material Design. Необходимо реализовать интерактивные элементы списка сообщений, включая отображение аватара, имени автора и возможность ставить отметку «лайк». Также требуется настроить фоновую синхронизацию сообщений с использованием WorkManager и отправку уведомлений при успешном обновлении данных. Приложение должно корректно работать в фоновом режиме, запрашивать необходимые разрешения при запуске и корректно реагировать на изменение состояния сети, сохраняя архитектуру MVVM.

Решение

Кастомный список сообщений:

Цветные аватары с первой буквой имени

Имя пользователя, текст сообщения

Лайки/дизлайки с изменяемыми счетчиками

Счетчик просмотров

Material Design компоненты:

MaterialCardView для каждого сообщения

FloatingActionButton для обновления данных

AppBarLayout с Toolbar

Лайки/дизлайки:

Клик по иконке меняет состояние (filled/border)

Счетчики увеличиваются/уменьшаются

Взаимоисключающие (лайк убирает дизлайк и наоборот)

WorkManager:

Фоновая синхронизация каждые 15 минут

Работает только при наличии интернета

Сохраняет пользовательские лайки/дизлайки при обновлении

Уведомления:

NotificationChannel для Android 8+

Уведомление "Новые данные получены" после синхронизации

Работает в фоновом режиме

Запросы

При входе в приложение запрашивает доступ на получения списка контактов и отправка уведомлений

6. Листинг кода

Main Activity

```
package com.example.messenger

import android.Manifest
import android.content.IntentFilter
import android.content.pm.PackageManager
import android.net.ConnectivityManager
import android.os.Build
import android.os.Bundle
import android.util.Log
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat
import androidx.navigation.fragment.NavHostFragment
import androidx.navigation.ui.setupWithNavController
import com.example.messenger.utils.NetworkReceiver
import
com.google.android.material.bottomnavigation.BottomNavigationView

class MainActivity : AppCompatActivity() {

    private val TAG = "MainActivity"
    private val networkReceiver = NetworkReceiver()
    private val PERMISSION_REQUEST_CODE = 100

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate: Activity создана")

        setContentView(R.layout.activity_main)

        setupNavigation()
        requestPermissions()
        registerNetworkReceiver()
    }

    private fun setupNavigation() {
        val navHostFragment = supportFragmentManager
```

```

        .findFragmentById(R.id.nav_host_fragment) as
NavHostFragment
        val navController = navHostFragment.navController

        val bottomNav =
findViewById<BottomNavigationView>(R.id.bottom_navigation)
        bottomNav.setupWithNavController(navController)
    }

    private fun requestPermissions() {
        val permissionsToRequest = mutableListOf<String>()

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS)
                != PackageManager.PERMISSION_GRANTED) {

permissionsToRequest.add(Manifest.permission.POST_NOTIFICATIONS)
                Log.d(TAG, "Добавлено разрешение POST_NOTIFICATIONS
для запроса")
            }
        }

        if (ContextCompat.checkSelfPermission(this,
Manifest.permission.READ_CONTACTS)
            != PackageManager.PERMISSION_GRANTED) {

permissionsToRequest.add(Manifest.permission.READ_CONTACTS)
                Log.d(TAG, "Добавлено разрешение READ_CONTACTS для
запроса")
            }

        if (permissionsToRequest.isNotEmpty()) {
            Log.d(TAG, "Запрос разрешений:
${permissionsToRequest.joinToString()}")
            ActivityCompat.requestPermissions(
                this,
                permissionsToRequest.toTypedArray(),
                PERMISSION_REQUEST_CODE
            )
        } else {
            Log.d(TAG, "Все разрешения уже предоставлены")
        }
    }

    private fun registerNetworkReceiver() {
        val filter =
IntentFilter(ConnectivityManager.CONNECTIVITY_ACTION)
    }

```

```

        registerReceiver(networkReceiver, filter)
        Log.d(TAG, "NetworkReceiver зарегистрирован")
    }

    override fun onRequestPermissionsResult(
        requestCode: Int,
        permissions: Array<out String>,
        grantResults: IntArray
    ) {
        super.onRequestPermissionsResult(requestCode, permissions,
grantResults)

        if (requestCode == PERMISSION_REQUEST_CODE) {
            permissions.forEachIndexed { index, permission ->
                if (grantResults[index] ==
PackageManager.PERMISSION_GRANTED) {
                    Log.d(TAG, "Разрешение предоставлено:
$permission")
                } else {
                    Log.d(TAG, "Разрешение отклонено: $permission")
                }
            }

            val allGranted = grantResults.all { it ==
PackageManager.PERMISSION_GRANTED }
            if (allGranted) {
                Toast.makeText(this, "Все разрешения
предоставлены", Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(this, "Некоторые разрешения
отклонены", Toast.LENGTH_SHORT).show()
            }
        }
    }

    override fun onDestroy() {
        super.onDestroy()
        try {
            unregisterReceiver(networkReceiver)
        } catch (e: Exception) {
            Log.e(TAG, "Ошибка отмены регистрации receiver", e)
        }
        Log.d(TAG, "onDestroy: Activity уничтожена")
    }
}

```


[FeedFragment.kt](#)

```
package com.example.messenger.ui.feed

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ProgressBar
import androidx.fragment.app.Fragment
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.lifecycleScope
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.messenger.R
import
com.google.android.material.floatingactionbutton.FloatingActionBut
ton
import kotlinx.coroutines.launch

class FeedFragment : Fragment() {

    private lateinit var viewModel: FeedViewModel
    private lateinit var adapter: MessageAdapter

    private lateinit var recyclerView: RecyclerView
    private lateinit var fab: FloatingActionButton
    private lateinit var progressBar: ProgressBar

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_feed, container,
false)
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        viewModel = ViewModelProvider(
            this,
            ViewModelProvider.AndroidViewModelFactory.getInstance(requireActiv
ity().application)
```

```

        ).get(FeedViewModel::class.java)

        recyclerView = view.findViewById(R.id.message_list)
        fab = view.findViewById(R.id.fab_refresh)
        progressBar = view.findViewById(R.id.progress_bar)

        setupRecyclerView()
        setupFab()
        observeViewModel()
    }

    private fun setupRecyclerView() {
        adapter = MessageAdapter(
            onLikeClick = { message ->
                viewModel.toggleLike(message)
            },
            onDislikeClick = { message ->
                viewModel.toggleDislike(message)
            }
        )

        recyclerView.layoutManager =
LinearLayoutManager(requireContext())
        recyclerView.adapter = adapter
    }

    private fun setupFab() {
        fab.setOnClickListener {
            Log.d("FeedFragment", "FAB нажата - обновление")
            viewModel.refresh()
        }
    }

    private fun observeViewModel() {
        viewLifecycleOwner.lifecycleScope.launch {
            viewModel.messages.collect { messages ->
                Log.d("FeedFragment", "Получено ${messages.size}
сообщений")

                adapter.updateData(messages)
            }
        }

        viewLifecycleOwner.lifecycleScope.launch {
            viewModel.isLoading.collect { isLoading ->
                progressBar.visibility = if (isLoading) View.VISIBLE
else View.GONE
            }
        }
    }

```

```
}  
}
```

FeedViewModel

```
package com.example.messenger.ui.feed  
  
import android.app.Application  
import android.util.Log  
import androidx.lifecycle.AndroidViewModel  
import androidx.lifecycle.viewModelScope  
import com.example.messenger.data.AppDatabase  
import com.example.messenger.data.Message  
import com.example.messenger.data.MessageRepository  
import com.example.messenger.data.RetrofitInstance  
import kotlinx.coroutines.flow.MutableStateFlow  
import kotlinx.coroutines.flow.StateFlow  
import kotlinx.coroutines.flow.asStateFlow  
import kotlinx.coroutines.launch  
  
class FeedViewModel(application: Application) :  
    AndroidViewModel(application) {  
    private val repository = MessageRepository(  
        RetrofitInstance.api,  
        AppDatabase.getDatabase(application)  
    )  
  
    private val _messages =  
        MutableStateFlow<List<Message>>(emptyList())  
    val messages: StateFlow<List<Message>> = _messages.asStateFlow()  
  
    private val _isLoading = MutableStateFlow(false)  
    val isLoading: StateFlow<Boolean> = _isLoading.asStateFlow()  
  
    private val limit = 20  
  
    init {  
        observeMessages()  
        loadMessages()  
    }  
  
    private fun observeMessages() {  
        viewModelScope.launch {  
            repository.getMessagesFlow().collect { messages ->  
                _messages.value = messages  
            }  
        }  
    }  
}
```

```

    }

    fun loadMessages() {
        viewModelScope.launch {
            try {
                _isLoading.value = true
                Log.d("FeedViewModel", "Загрузка сообщений")
                repository.fetchMessages(limit)
                Log.d("FeedViewModel", "Сообщения загружены")
            } catch (e: Exception) {
                Log.e("FeedViewModel", "Ошибка загрузки сообщений",
e)

            } finally {
                _isLoading.value = false
            }
        }
    }

    fun toggleLike(message: Message) {
        viewModelScope.launch {
            try {
                repository.toggleLike(message)
                Log.d("FeedViewModel", "Лайк переключен для
сообщения ${message.id}")
            } catch (e: Exception) {
                Log.e("FeedViewModel", "Ошибка переключения лайка",
e)

            }
        }
    }

    fun toggleDislike(message: Message) {
        viewModelScope.launch {
            try {
                repository.toggleDislike(message)
                Log.d("FeedViewModel", "Дизлайк переключен для
сообщения ${message.id}")
            } catch (e: Exception) {
                Log.e("FeedViewModel", "Ошибка переключения
дизлайка", e)
            }
        }
    }

    fun refresh() {
        Log.d("FeedViewModel", "Обновление данных")
        loadMessages()
    }
}

```

```
}
```

MessageAdapter

```
package com.example.messenger.ui.feed
```

```
import android.graphics.Color
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ImageView
import android.widget.TextView
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.RecyclerView
import com.example.messenger.R
import com.example.messenger.data.Message

class MessageAdapter(
    private val onLikeClick: (Message) -> Unit,
    private val onDislikeClick: (Message) -> Unit
) : RecyclerView.Adapter<MessageAdapter.MessageViewHolder>() {

    private var items: List<Message> = emptyList()

    class MessageViewHolder(view: View) :
        RecyclerView.ViewHolder(view) {
        val avatar: TextView = view.findViewById(R.id.item_avatar)
        val userName: TextView =
            view.findViewById(R.id.item_user_name)
        val messageText: TextView =
            view.findViewById(R.id.item_message_text)
        val likeIcon: ImageView =
            view.findViewById(R.id.item_like_icon)
        val likeCount: TextView =
            view.findViewById(R.id.item_like_count)
        val dislikeIcon: ImageView =
            view.findViewById(R.id.item_dislike_icon)
        val dislikeCount: TextView =
            view.findViewById(R.id.item_dislike_count)
        val viewCount: TextView =
            view.findViewById(R.id.item_view_count)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType:
        Int): MessageViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_message, parent, false)
        return MessageViewHolder(view)
    }
}
```

```

    }

    override fun onBindViewHolder(holder: MessageViewHolder,
position: Int) {
        val item = items[position]

        val avatarColors = listOf(
            "#E91E63", "#9C27B0", "#3F51B5", "#2196F3", "#009688",
            "#4CAF50", "#FF9800", "#FF5722", "#795548", "#607D8B"
        )
        val colorIndex = position % avatarColors.size

holder.avatar.setBackgroundColor(Color.parseColor(avatarColors[colorIndex]))

        val initial = item.name.firstOrNull()?.uppercase() ?: "?"
        holder.avatar.text = initial

        holder.userName.text = item.name
        holder.messageText.text = item.email

        val totalLikes = item.reactions.likes + item.userLikes
        val totalDislikes = item.reactions.dislikes +
item.userDislikes

        holder.likeCount.text = totalLikes.toString()
        holder.dislikeCount.text = totalDislikes.toString()
        holder.viewCount.text = item.views.toString()

        val likeIconRes = if (item.isLiked) {
            R.drawable.ic_favorite_filled
        } else {
            R.drawable.ic_favorite_border
        }
        holder.likeIcon.setImageResource(likeIconRes)

        val dislikeIconRes = if (item.isDisliked) {
            R.drawable.ic_thumb_down_filled
        } else {
            R.drawable.ic_thumb_down
        }
        holder.dislikeIcon.setImageResource(dislikeIconRes)

        holder.likeIcon.setOnClickListener {
            onLikeClick(item)
        }

        holder.dislikeIcon.setOnClickListener {

```

```

        onDislikeClick(item)
    }
}

override fun getItemCount(): Int = items.size

fun updateData(newItems: List<Message>) {
    val diffCallback = MessageDiffCallback(items, newItems)
    val diffResult = DiffUtil.calculateDiff(diffCallback)

    items = newItems
    diffResult.dispatchUpdatesTo(this)
}

private class MessageDiffCallback(
    private val oldList: List<Message>,
    private val newList: List<Message>
) : DiffUtil.Callback() {

    override fun getOldListSize() = oldList.size
    override fun getNewListSize() = newList.size

    override fun areItemsTheSame(oldItemPosition: Int,
newItemPosition: Int): Boolean {
        return oldList[oldItemPosition].id ==
newList[newItemPosition].id
    }

    override fun areContentsTheSame(oldItemPosition: Int,
newItemPosition: Int): Boolean {
        return oldList[oldItemPosition] ==
newList[newItemPosition]
    }
}
}

```

ProfileFragment

```

package com.example.messenger.ui.profile

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView
import androidx.fragment.app.Fragment

```

```
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.lifecycleScope
import com.example.messenger.R
import com.google.android.material.textfield.TextInputEditText
import kotlinx.coroutines.launch

class ProfileFragment : Fragment() {

    private val TAG = "ProfileFragment"
    private lateinit var viewModel: ProfileViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate: Fragment создан")
    }

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        Log.d(TAG, "onCreateView: View создан")
        return inflater.inflate(R.layout.fragment_profile,
container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "onViewCreated: View готов к использованию")

        viewModel =
ViewModelProvider(this).get(ProfileViewModel::class.java)

        val nameEditText =
view.findViewById<TextInputEditText>(R.id.profile_name_edit)
        val statusEditText =
view.findViewById<TextInputEditText>(R.id.profile_status_edit)
        val nameTextView =
view.findViewById<TextView>(R.id.profile_name)
        val statusTextView =
view.findViewById<TextView>(R.id.profile_status)
        val emailTextView =
view.findViewById<TextView>(R.id.profile_email)
        val phoneTextView =
view.findViewById<TextView>(R.id.profile_phone)
        val saveButton =
view.findViewById<Button>(R.id.save_button)
```



```

        viewLifecycleOwner.lifecycleScope.launch {
            viewModel.profileData.collect { profile ->
                nameTextView.text = profile.name
                statusTextView.text = profile.status
                emailTextView.text = profile.email
                phoneTextView.text = profile.phone
            }
        }

        saveButton.setOnClickListener {
            val newName = nameEditText.text?.toString()?.trim() ?:
""
            val newStatus = statusEditText.text?.toString()?.trim()
?: ""

            if (newName.isNotEmpty()) {
                viewModel.updateName(newName)
            }
            if (newStatus.isNotEmpty()) {
                viewModel.updateStatus(newStatus)
            }

            nameEditText.text?.clear()
            statusEditText.text?.clear()
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        Log.d(TAG, "onDestroyView: View уничтожен")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy: Fragment уничтожен")
    }
}

```

ProfileViewModel

```

package com.example.messenger.ui.profile

import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow

```

```

data class ProfileData(
    val name: String = "Якунин Андрей",
    val status: String = "В сети",
    val email: String = "andrucrut@mail.ru",
    val phone: String = "+79818203438"
)

class ProfileViewModel : ViewModel() {
    private val _profileData = MutableStateFlow(ProfileData())
    val profileData: StateFlow<ProfileData> =
        _profileData.asStateFlow()

    fun updateName(newName: String) {
        _profileData.value = _profileData.value.copy(name =
newName)
    }

    fun updateStatus(newStatus: String) {
        _profileData.value = _profileData.value.copy(status =
newStatus)
    }
}

```

SettingsFragment

```

package com.example.messenger.ui.settings

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment
import com.example.messenger.R
import com.google.android.material.switchmaterial.SwitchMaterial

class SettingsFragment : Fragment() {

    private val TAG = "SettingsFragment"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate: Fragment создан")
    }

    override fun onCreateView(

```

```

        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        Log.d(TAG, "onCreateView: View создан")
        return inflater.inflate(R.layout.fragment_settings,
container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "onViewCreated: View готов к использованию")

        val themeSwitch =
view.findViewById<SwitchMaterial>(R.id.theme_switch)
        themeSwitch.setOnCheckedChangeListener { _, isChecked ->
            if (isChecked) {
                AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_
YES)
                Log.d(TAG, "Темная тема включена")
            } else {
                AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_
NO)
                Log.d(TAG, "Светлая тема включена")
            }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        Log.d(TAG, "onDestroyView: View уничтожен")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy: Fragment уничтожен")
    }
}

```

[Message.kt](#)

```

package com.example.messenger.data

import androidx.room.Entity
import androidx.room.PrimaryKey

```

```

import androidx.room.TypeConverter
import androidx.room.TypeConverters
import com.google.gson.Gson
import com.google.gson.annotations.SerializedName

data class Reactions(
    val likes: Int = 0,
    val dislikes: Int = 0
)

class Converters {
    @TypeConverter
    fun fromReactions(reactions: Reactions?): String? {
        return reactions?.let { Gson().toJson(it) }
    }

    @TypeConverter
    fun toReactions(reactionsString: String?): Reactions? {
        return reactionsString?.let { Gson().fromJson(it,
Reactions::class.java) }
    }
}

@Entity(tableName = "messages")
@TypeConverters(Converters::class)
data class Message(
    @PrimaryKey
    val id: Int,

    @SerializedName("title")
    val name: String,

    @SerializedName("body")
    val email: String,

    @SerializedName("userId")
    val body: String,

    @SerializedName("reactions")
    val reactions: Reactions = Reactions(),

    @SerializedName("views")
    val views: Int = 0,

    val isLiked: Boolean = false,
    val isDisliked: Boolean = false,

    val userLikes: Int = 0,

```

```
    val userDislikes: Int = 0
)
```

Repository

```
package com.example.messenger.data

import android.util.Log
import kotlinx.coroutines.flow.Flow

class MessageRepository(private val api: MessageApiService,
private val db: AppDatabase) {
    private val dao = db.messageDao()

    fun getMessagesFlow(): Flow<List<Message>> {
        return dao.getAllMessagesFlow()
    }

    suspend fun fetchMessages(limit: Int): List<Message> {
        return try {
            Log.d("MessageRepository", "Загрузка из API: limit=$limit")

            val response = api.getMessages(limit, 0)
            Log.d("MessageRepository", "Получено ${response.posts.size} сообщений из API")

            val existingMessages = dao.getAllMessages().associateBy { it.id }

            val updatedMessages = response.posts.map { newMsg ->
                val existing = existingMessages[newMsg.id]
                if (existing != null) {
                    newMsg.copy(
                        isLiked = existing.isLiked,
                        isDisliked = existing.isDisliked,
                        userLikes = existing.userLikes,
                        userDislikes = existing.userDislikes
                    )
                } else {
                    newMsg
                }
            }

            dao.deleteAll()
            dao.insertMessages(updatedMessages)

            updatedMessages
        } catch (e: Exception) {
            Log.e("MessageRepository", "Ошибка загрузки сообщений из API: ${e.message}")
            emptyList()
        }
    }
}
```

```

        } catch (e: Exception) {
            Log.e("MessageRepository", "Ошибка загрузки из API,
используем кэш", e)

            val cachedMessages = dao.getAllMessages()
            Log.d("MessageRepository", "Загружено
${cachedMessages.size} сообщений из БД")

            cachedMessages
        }
    }

    suspend fun toggleLike(message: Message) {
        val newIsLiked = !message.isLiked
        val newUserLikes = if (newIsLiked) message.userLikes + 1
else message.userLikes - 1

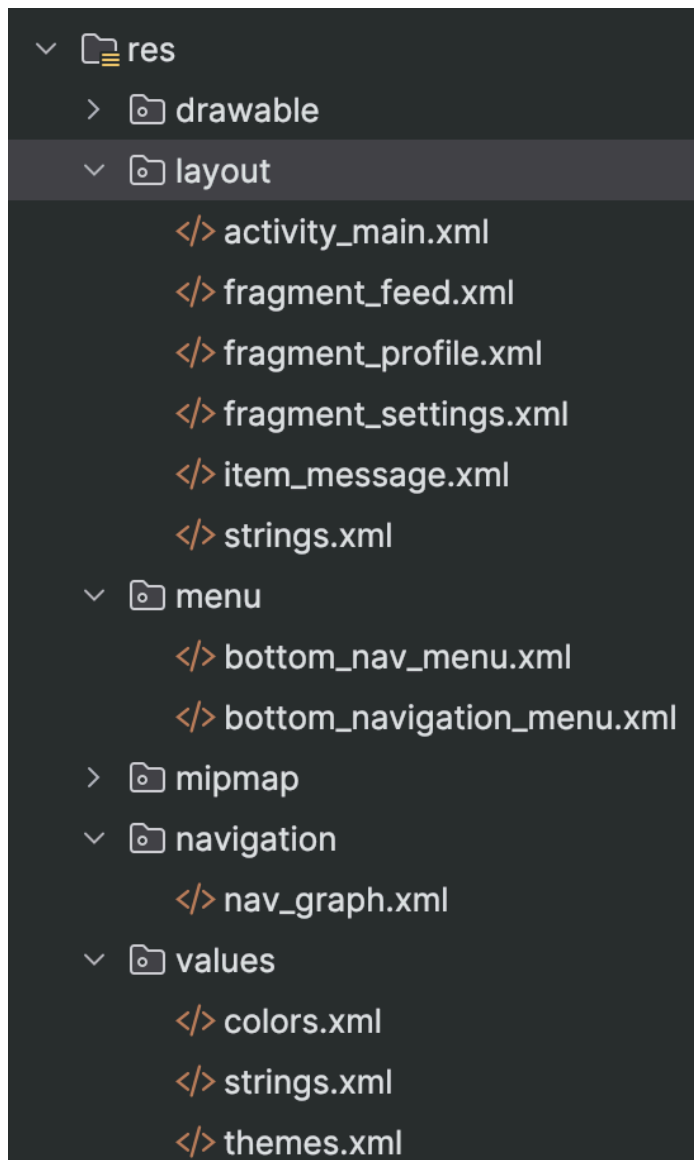
        val updatedMessage = message.copy(
            isLiked = newIsLiked,
            isDisliked = false,
            userLikes = newUserLikes,
            userDislikes = if (message.isDisliked)
message.userDislikes - 1 else message.userDislikes
        )
        dao.updateMessage(updatedMessage)
    }

    suspend fun toggleDislike(message: Message) {
        val newIsDisliked = !message.isDisliked
        val newUserDislikes = if (newIsDisliked)
message.userDislikes + 1 else message.userDislikes - 1

        val updatedMessage = message.copy(
            isDisliked = newIsDisliked,
            isLiked = false,
            userDislikes = newUserDislikes,
            userLikes = if (message.isLiked) message.userLikes - 1
else message.userLikes
        )
        dao.updateMessage(updatedMessage)
    }
}

```

7. xml архитектура



8. Вывод по работе

В процессе выполнения курсовой работы было разработано Android-приложение, основанное на архитектуре MVVM с применением компонентов Material Design и актуальных инструментов платформы Android, таких как Room, Retrofit, Navigation Component, LiveData и ViewModel. Реализована полноценная работа с данными, включающая загрузку информации из сети, её локальное хранение и последующее отображение в пользовательском интерфейсе с поддержкой интерактивных функций. Разработанное приложение может быть использовано как базовая платформа для

дальнейшего расширения функционала, включая реализацию фоновой синхронизации, системы уведомлений и дополнительных возможностей работы с профилем. Результаты работы подтверждают освоение основных принципов и практик разработки современных Android-приложений